Requirements

a. Set of Requirements Designated for Phase 1

Old Requirements:

Feature	Phase	Task	Description	Technologies
Accounts	1	Create Account	Allow users to create accounts, and specify whether they are a customer or employee, allowing different permissions	Javascript, SQL
Accounts	3	Update Account	Allow the user to update account information or delete the account	Javascript, SQL
Menu	1	Add items to the menu	Allow employees to add items to the menu	Javascript, SQL, HTML
Menu	1	Remove items from the menu	Allow employees to remove items from the menu	Javascript, SQL, HTML
Menu	3	Add deals to the menu	Allow for deals to be shown on specific menu items	Javascript, SQL, HTML
Cart	1	Add items to cart	Allow customers to add items to their individual cart	Javascript, HTML
Cart	1	Remove items from cart	Allow customers to remove items from their individual cart	Javascript, HTML
Orders	2	Place order	Allow customers to place an order with the items in their cart	Javascript, HTML
Orders	2	Track order	Allow customers to track the status of their order	Javascript
Orders	3	Search for Order	Allow employees to search orders by ID	SQL
Orders	2	Update Order	Allow employees to change the status of orders	SQL

Inventory	2	Track Inventory	Allow employees to view what is currently in the inventory of the restaurant	HTML

What was changed: We moved some of the requirements to different phases and ended up not using an SQL database, instead opting for using localstorage in the Javascript. That way we can keep the program within its own closed system.

What was moved: Create Account, Add deals to menu, Place Order, Search for Order, Update Order, Track Inventory

Create Account 1 -> 2 - This was not as important to ensuring that the employee accounts worked properly, so we decided to push it back and focus more on the functionality of the account rather than being able to make them at the moment

Add deals to menu 3 -> 1 - This was very simple to set up, and we placed simple placeholders that are not fully integrated, but will be finalized in stage 2.

Place Order 2 -> 1 - Placing orders was a core functionality to the program so we thought it was very important to move this forward so that way we can ensure that the main function of the program is working.

Search for order 3 -> 1 with the order placing already established it was easy to continue the logic and implement the search function for employees, this way we can build off of this for our updating orders feature

Track Inventory 2 -> 1 - The inventory was moved forward because we had some extra time and decided to get it completed before starting the next phase.

Feature	Phase	Task	Description	Technologies
Accounts	2 (1)	Create Account	Allow users to create accounts, and specify whether they are a customer or employee, allowing different permissions	Javascript
Accounts	3	Update Account	Allow the user to update account information or delete the account	Javascript
Menu	1	Add items to the menu	Allow employees to add items to the menu	Javascript, HTML
Menu	1	Remove items	Allow employees to remove	Javascript,

		from the menu	items from the menu	HTML
Menu	1 (3)	Add deals to the menu	Allow for deals to be shown on specific menu items	Javascript, HTML
Cart	1	Add items to cart	Allow customers to add items to their individual cart	Javascript, HTML
Cart	1	Remove items from cart	Allow customers to remove items from their individual cart	Javascript, HTML
Orders	1 (2)	Place order	Allow customers to place an order with the items in their cart	Javascript, HTML
Orders	2	Track order	Allow customers to track the status of their order	Javascript
Orders	1 (3)	Search for Order	Allow employees to search orders by ID	Javascript
Orders	3 (2)	Update Order	Allow employees to change the status of orders	Javascript
Inventory	1 (2)	Track Inventory	Allow employees to view what is currently in the inventory of the restaurant	HTML

Detailed Requirements (Phase 1):

Menu and Cart

- The system will allow customers to add items to a cart.
- Display all available menu items along with their prices so customers can easily make a selection.
- Allows customers to remove or update items in the cart before placing an order.
- Allow Customers to make specific customizations to items before placing an order.
- The system will display highlighted items and discounts from the menu on the landing page, labeled as “HOT” and “DEALS.”

Order placement and Tracking

- Allows customers to place an order after confirming their cart.
- It checks inventory before confirming the order.
- System shall generate a unique order ID for the order, if ordering is successful.
- If ordering is not successful, system should notify the customer.

Order Manager

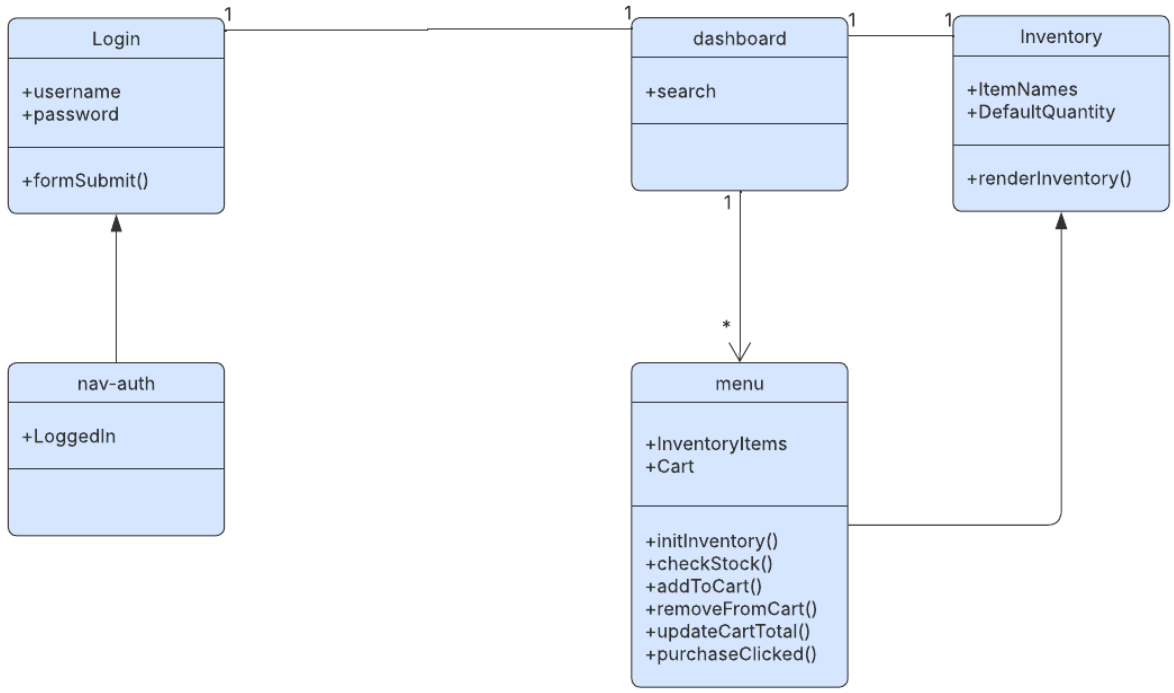
- Gives staff access to the Order Manager.
- System should allow employees to search for orders using their unique order ID.

Inventory Tracker

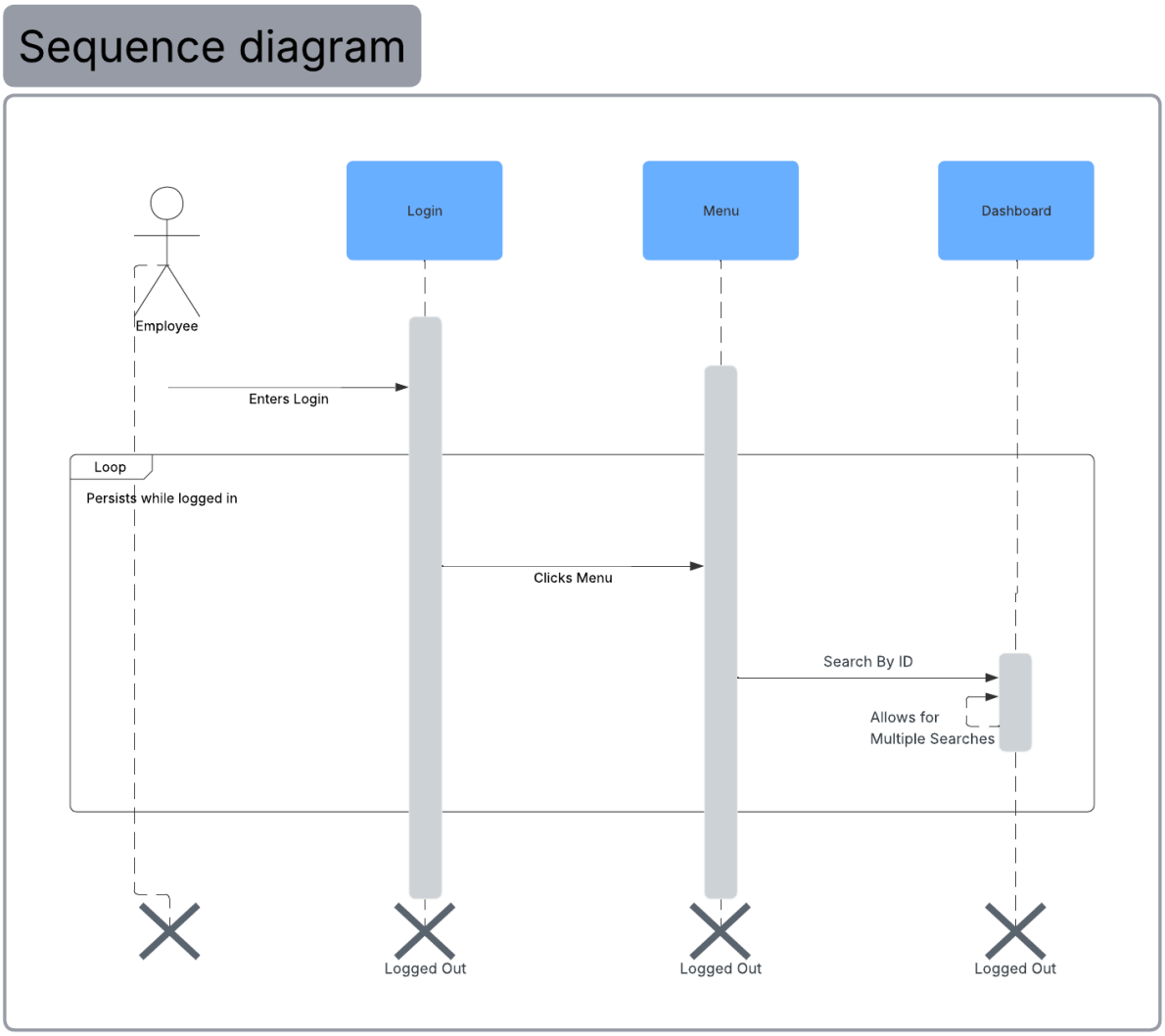
- The employee interface will give staff access to the inventory tracker.
- It will track item quantities in inventory.
- Inventory quantities will update automatically when an order is placed.
- The system should reject the ordering of items that are marked out of stock.

b. UML Design

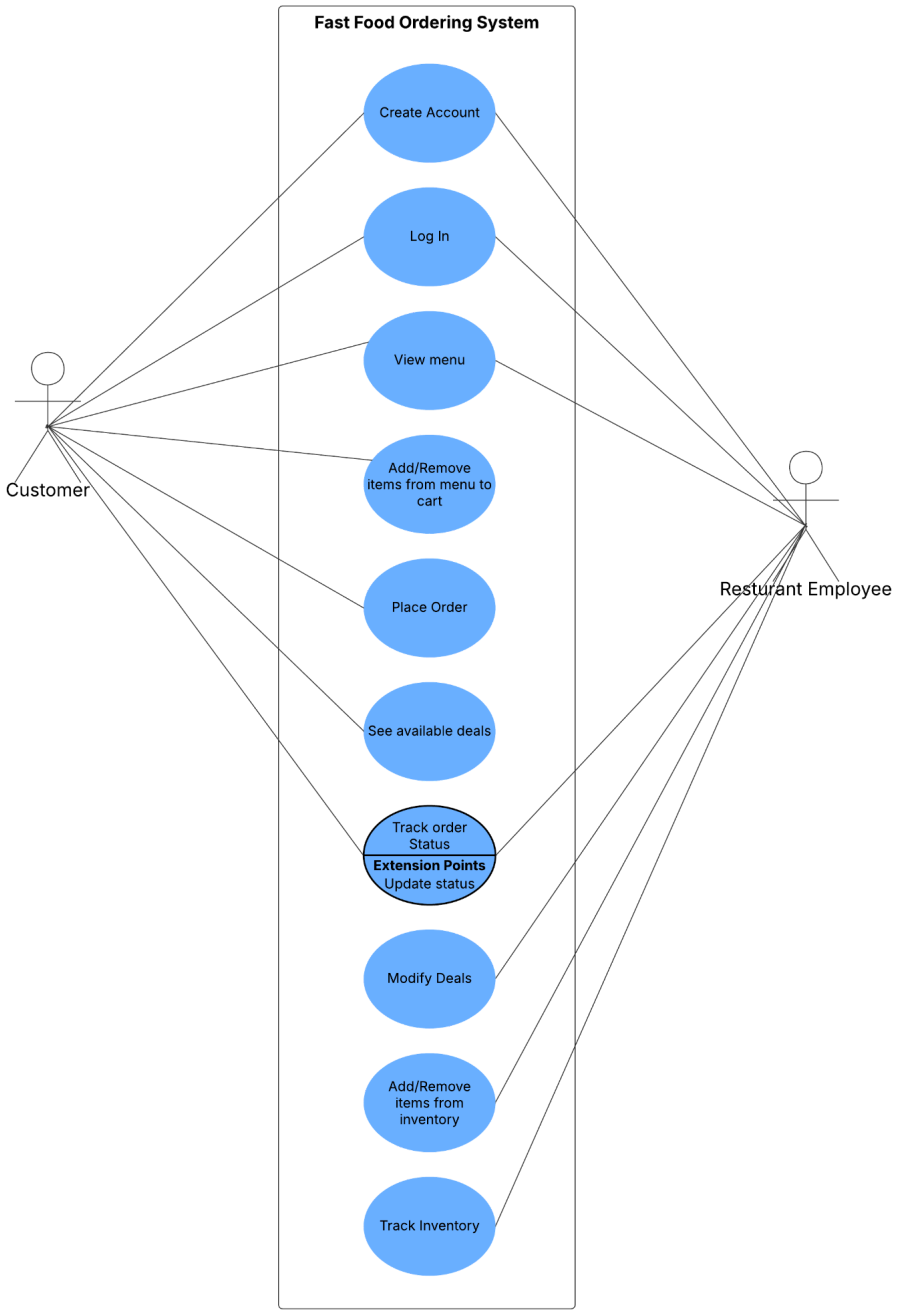
Class diagram



Sequence diagram



Use case diagram



c. User Manual

Customer Manual:

1) Open the Quadrant One Food Ordering webpage and locate the “Menu” option in the navigation bar.
2) Select and add your desired menu items with their respective “Add to Cart” button. Cart should update as you do.
3) Inside the cart below, you can edit the amount of each item in the "Quantity" section and remove items with the “Remove” button.
4) Review your cart and select “Purchase” to place your order.

Employee Manual:

1) Open the Quadrant One Food Ordering webpage and locate the “Employee Login” option in the navigation bar.
2) Login in with your employee ID. (Ex: User: employee, Password: 1234)
3) You will be redirected to the Order Search Dashboard. Here you can search for Order ID’s, returning a search when pressing the button labeled “Search”.
4) You can also view Inventory by selecting the navigation bar button labeled “Inventory”.
5) Inside of the Inventory, update the inventory by selecting “Update”. You can also update the amount of inventory you are currently updating in the "Quantity" column.

d. Instructions & Test Cases

How to Compile/Run

This project does not need to be compiled in its current state. It is a static frontend only. To begin, unzip the project (or pull from the repo) and open index.html in your browser. This is the starting point and other pages can be accessed from here.

Test Cases

Test Scenario 1: Employee Login
Description: Attempt employee login with empty or correct credentials.
Functionality: Tests login form validation for required fields.
Case 1: Blank

Navigate to the Employee Login Page.

Input: Username and password blank.

Output: “Please fill in all fields”.

Case 2: Correct

Input: Username (*employee*) and password (*1234*) correct.

Output: Page grants access to employee dashboard.

Test Scenario 2: Cart Functionality

Description: Attempt to add, change the quantity of, and remove items from the cart. This scenario is a continuation of Scenario 1.

Functionality: Tests the cart logic and rendering.

Case 1: Add

Navigate to the Menu page.

Input: Click Add to Cart button in menu.

Output: Item appears in cart with quantity, price, and Remove button.

Case 2: Edit

Input: Click the up or down arrows on the existing item in the cart.

Output: Item quantity and total price will change.

Case 3: Remove

Input: Click Remove button with existing item in cart.

Output: Item will disappear from cart.

Case 4: Purchase

Continue from Case 1

Input: Click Purchase button with existing item in cart.

Output: “Thank you for your purchase! Your order number is (order number).”

Test Scenario 3: Viewing Orders

Description: Attempt to view existing and nonexistent orders from Search Dashboard. This scenario is a continuation of Scenario 2.

Functionality: Tests the order ID functionality.

Case 1: Search Existing Order

Navigate to Search Dashboard.

Input: Any previous existing order ID.

Expected Output: Order number, name of items in order, number of items in order.

Actual Output: Order number, name of items in order.

Case 2: Search Nonexistant Order

Input: Any nonexistent order number (example: 999999)

Output: “No Order found for ID: 999999”

Case 2: Search String

Input: Any string (example: hello world)

Expected Output: message warning user to enter a number

Actual Output: “No Order found for ID: hello world”

e. Reflection

A brief reflection on what has been accomplished, what went well and could be Improved.

Accomplished:
The core functionalities of each requirement has been implemented, with some changes. The HTML skeleton is almost fully complete. The CSS styling and JS logic are both in progress, but overall the application is mostly functional.

Went Well:
The entire group maintained very consistent and clear communication. We have gotten very familiar with GitHub and can now manage our project seamlessly through it. Progress is going quickly. All members have contributed. The group adapted quickly to changing requirements.

Needs Improvement:
Although our communication over text was consistent and effective, we did not meet over call as many times as we had planned. This led to completions that did not align with our pre-planned schedule.

f. Member contribution table

(should describe who wrote what components or classes of the system and what parts of the report). Add more rows as needed.

Member name	Contribution description	Overall Contribution (%)	Note (if applicable)
Kenny Lee	Inventory Tracking and Page, CSS Styling	16.67%	
Cody Howell	Search Dashboard, CR Login for Dashboard, CSS styling (Menu, Cart, Login),	16.67%	
Alena Emmanuelli	Menu HTML, CSS styling (Navbar, Menu), d. Instructions and Test Cases, e. Reflection	16.67%	

Kyle Burchett	Cart and Ordering Logic. UML diagrams. Requirements	16.67%	
Jayden Patton	Menu HTML	16.67%	
Taying Yang (Tai Yang)	Dashboard and Login	16.67%	